

Allotiq

Smart Resource Allocation System — project plan

Every request. The right room. Even when plans change.

Team: Aaditya (Intelligence) · Aditi (Platform & workflow) · Nikhil (Experience, analytics & deployment) · 29 September 2026

Room finders answer "what's free?". Allotiq answers "what's the best arrangement for everyone?" and keeps it optimal through no-shows, maintenance and priority requests.

Context

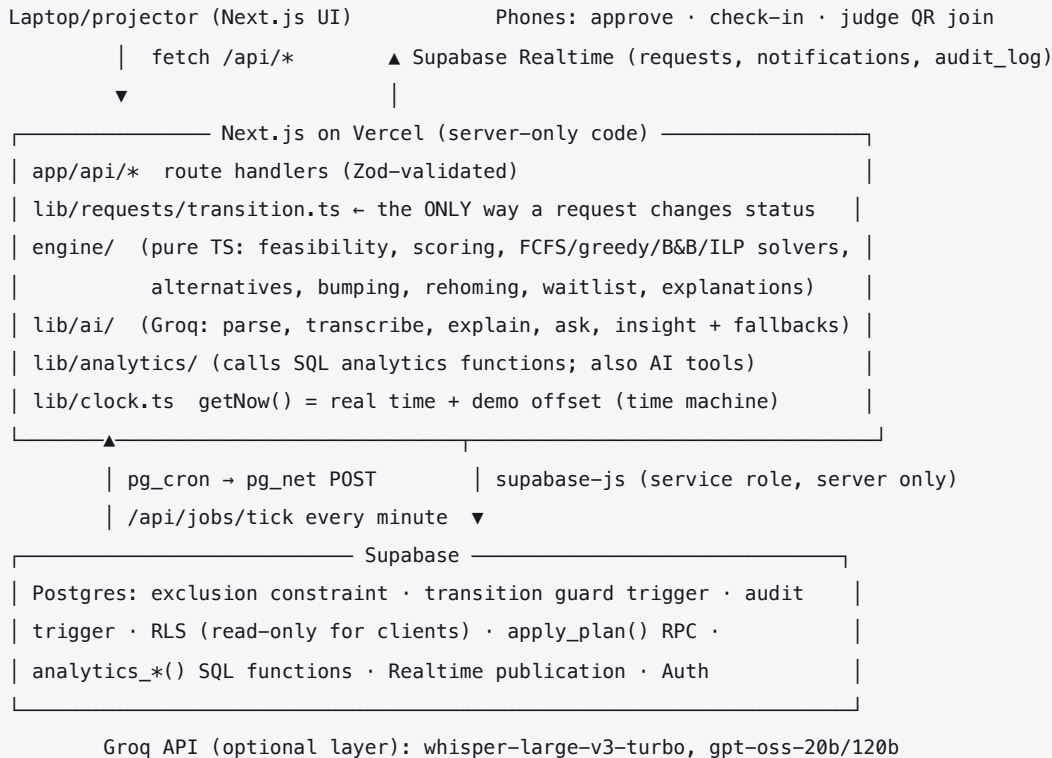
- Next round of Vision2Web 2026 (Category 2, Innovation 2). The team is Aaditya, Aditi and Nikhil. We need a deployed, demo-ready prototype.
- Judges score practicality, innovation, working functionality, UX and problem-solving. The goal is for them to leave saying "this is unique, and that demo was great."
- The build window is short (under a week), but scope stays full.** What keeps it safe is **phase order**: the 5-scene demo path works end to end on the deployed URL at the end of Phase 3, and only then do the wow layer, polish and rehearsal begin.
- Stack: **Next.js (App Router, TypeScript) + Supabase (Postgres, Auth, Realtime, pg_cron) + Groq**, deployed on **Vercel**. The team knows React but is new to Next.js and Supabase, so the plan includes snippets and gotchas.
- Workflow: each member owns a vertical slice, pushes to GitHub, and others pull. The repo layout is split by owner so merges rarely conflict.
- Language: **English only** (text and voice). Tamil and other Indian languages are a roadmap item.
- This is a new project with nothing built yet. **Nikhil creates the repo.**
- UI design references come from Nikhil** when frontend work starts. This plan fixes behaviour and content, and the look follows those references.

Why this stack (free, and each piece earns its place)

Need	Choice	Why
Double-booking prevented in the DB	Postgres <code>EXCLUDE USING gist</code>	Only Postgres does this natively. Firebase and Mongo can't.
"Judge approves on phone → projector updates live"	Supabase Realtime	The wow moment for free, with no socket server to build.
Hold expiry and no-show jobs every minute	Supabase <code>pg_cron</code> + <code>pg_net</code> calling <code>/api/jobs/tick</code>	Vercel's free (Hobby) plan only runs cron jobs once a day.
One language for engine, API and UI	TypeScript everywhere	Engine types are shared with the UI; one deploy.
Auth with roles	Supabase Auth + a <code>profiles.role</code> column	Built in; no extra service.
Hosting	Vercel Hobby (app) + 2 Supabase free projects (<code>allotiq-dev</code> , <code>allotiq-demo</code>)	\$0. The demo DB stays clean and reseedable.

Libraries: Tailwind + shadcn/ui, lucide icons, Recharts (charts), Motion (formerly framer-motion; animations), `date-fns` + `@date-fns/tz` (IST), Zod, `qrcode.react`, `chrono-node` (fallback date parsing when Groq is down), `groq-sdk`, Vitest, and **stretch**: `highs` (the HiGHS ILP solver compiled to WebAssembly).

1. Architecture



Core rules (also written into `docs/CONTRACTS.md` so everyone follows them):

1. Browsers never write to the DB directly. All writes go through `/api/*` using the service role, and authorisation is checked in code (`requireRole`). RLS is only for Realtime and read access.
2. Every status change goes through `transition(requestId, to, {actor, note})` . The DB trigger rejects illegal transitions and writes the audit row, so no code path can skip either.
3. Business time always comes from `getNow()` . Never use `Date.now()` or the DB's `now()` in logic; the demo time machine depends on this.
4. Intervals are half-open `[start, end)` and stored as `timestampz` . Everything is displayed in Asia/Kolkata.
5. The LLM never decides an allocation. It only parses, phrases, and picks which safe query to run.

Repo layout (owners in brackets; `contracts/` is shared and frozen)

```

allotiq/
  docs/CONTRACTS.md          # contracts summary + rules above + ownership map
  .github/workflows/ci.yml    # typecheck + lint + vitest on every PR [Nikhil]
  supabase/migrations/*.sql   # schema, triggers, RLS, RPCs [Aditi]
  supabase/analytics/*.sql    # analytics_* functions [Nikhil]
  scripts/seed/               # catalog + 4-week history generator [Nikhil]; scenarios [Aaditya, Aditi]
  src/contracts/              # domain.ts, engine.ts, api.ts, ai.ts - SHARED; changes need a PR all 3
approve
  src/engine/**               # pure TS, no Next/DB imports [Aaditya]
  src/server/engine-adapter.ts # DB rows → EngineContext; plans → apply_plan [Aaditya]
  
```

```

src/lib/ai/**                # Groq + fallbacks                [Aaditya]
src/lib/{db,auth,clock,requests,notify,jobs}/**                [Aditi]
src/lib/analytics/**          [Nikhil]
src/components/ui/**, src/components/kit/**  # design system (from Nikhil's refs) [Nikhil]
src/app/(auth)/login, /join          [Aditi]
src/app/r/** (new request, my bookings, status), src/app/availability [Nikhil]
src/app/approvals/**, src/app/c/[roomCode] (check-in)          [Aditi]
src/app/admin/{resources,audit,demo}          [Aditi]
src/app/admin/{lab,disruptions}              [Aaditya]
src/app/admin/dashboard                  [Nikhil]

```

Git workflow

- **Nikhil creates the GitHub repo and adds Aaditya and Aditi as collaborators.** Everyone commits and pushes their own work.
- Branches are `aaditya/...`, `aditi/...`, `nikhil/...`. Merge to `main` via PR **small and often** (at least every few hours) and at the end of every phase.
- Before pushing, run `git pull --rebase origin main`. Never force-push to `main`.
- CI (typecheck + engine tests) must be green. Vercel gives every PR a preview URL, and `main` is always the live URL.
- Only Aditi runs `supabase db push`. Anyone else who needs a schema change asks her or opens a PR against `supabase/migrations`.
- After each migration, run `npm run supabase gen types typescript` and commit the output to `src/lib/db/types.gen.ts`.

2. Database schema (`supabase/migrations/0001_core.sql` and onwards)

```

create extension if not exists btree_gist;
create extension if not exists pg_cron;  -- if this errors, enable it in Dashboard → Database → Extensions
create extension if not exists pg_net;

create type user_role      as enum ('requester','approver','admin');
create type requester_kind as enum ('faculty','club','department','student');
create type room_type      as enum ('lab','classroom','seminar_hall','meeting_room','auditorium');
create type access_rule    as enum ('open','dept_only');
create type purpose_kind   as enum
('exam','academic','department_event','club_event','meeting','student_activity');
create type request_status as enum ('waitlisted','pending','approved','checked_in','completed',
                                     'rejected','expired','cancelled','auto_released','bumped');

create table buildings (id uuid primary key default gen_random_uuid(), code text unique not null,
                        name text not null, lat double precision, lng double precision);
create table departments (id uuid primary key default gen_random_uuid(), code text unique not null,
                          name text not null, building_id uuid references buildings);
create table profiles (
  id uuid primary key references auth.users on delete cascade,
  full_name text not null, role user_role not null default 'requester',
  kind requester_kind, department_id uuid references departments, org_name text  -- club name etc.
);

create table rooms (

```

```

    id uuid primary key default gen_random_uuid(),
    code text unique not null, name text not null,
    building_id uuid not null references buildings,
    type room_type not null,
    capacity int not null check (capacity > 0),
    systems_count int not null default 0 check (systems_count >= 0),
    features text[] not null default '{}',           -- projector, mic, computers, smart_board, ac, ...
    department_id uuid references departments,
    access access_rule not null default 'open',
    approver_id uuid references profiles,
    open_time time not null default '08:00', close_time time not null default '20:00',
    open_days smallint[] not null default '{1,2,3,4,5,6}', -- ISO weekdays
    attributes jsonb not null default '{}',          -- generic resource model → roadmap:
vehicles/equipment
    qr_secret text not null default encode(gen_random_bytes(12),'hex'),
    is_active boolean not null default true,
    check (close_time > open_time)
);

create table room_blackouts (
    id uuid primary key default gen_random_uuid(),
    room_id uuid not null references rooms on delete cascade,
    during tstzrange not null check (not isempty(during)),
    reason text not null, created_by uuid references profiles, created_at timestamptz not null default now()
);

create index on room_blackouts using gist (room_id, during);

create table requests (
    id uuid primary key default gen_random_uuid(),
    requester_id uuid not null references profiles,
    title text not null, purpose purpose_kind not null,
    priority smallint not null,                      -- derived server-side from purpose; never from client input
    headcount int not null check (headcount between 1 and 5000),
    min_systems int not null default 0,
    required_features text[] not null default '{}',
    room_type room_type, preferred_building_id uuid references buildings,
    during tstzrange not null check (lower(during) < upper(during)), -- default bounds are '[' = half-open
    room_id uuid references rooms,
    status request_status not null default 'pending',
    hold_expires_at timestamptz, checked_in_at timestamptz,
    decided_by uuid references profiles, decision_reason text,
    unplaced_reason text,                            -- engine couldn't place it → feeds "unmet demand"
    offered_alternatives jsonb, score_breakdown jsonb,
    source text not null default 'form' check (source in ('form','text','voice','lab','seed')),
    raw_input text,
    last_actor_id uuid references profiles, last_action text, last_action_at timestamptz, -- read by the audit
trigger
    created_at timestamptz not null, updated_at timestamptz not null default now(),      -- app sets created_at
= getNow()
    constraint room_needed check (room_id is not null or status in
('waitlisted','rejected','expired','cancelled','bumped')),
    -- ★ THE anti-double-booking guarantee. Deferrable so apply_plan() can swap rooms inside one transaction.

```

```

    constraint no_double_booking exclude using gist (room_id with =, during with &&)
    where (status in ('pending','approved','checked_in')) deferrable initially deferred
);
create index on requests (status, lower(during));
create index on requests (requester_id);

create table audit_log (
    id bigint generated always as identity primary key,
    at timestamptz not null, actor_id uuid, entity text not null, entity_id uuid not null,
    action text not null, from_status request_status, to_status request_status,
    details jsonb not null default '{}'
);
create table notifications (
    id uuid primary key default gen_random_uuid(), user_id uuid not null references profiles,
    kind text not null, title text not null, body text, request_id uuid references requests,
    created_at timestamptz not null, read_at timestamptz
);
create table app_settings (key text primary key, value jsonb not null);
-- clock_offset_ms, demo_anchor, engine_weights, hold_minutes (120), noshow_grace_minutes (15)
create table demo_tokens (token text primary key, user_id uuid references profiles, expires_at timestamptz,
used_count int default 0);
create table lab_scenarios (id text primary key, name text, description text, requests jsonb not null);
create table engine_runs (id uuid primary key default gen_random_uuid(), kind text, solver text,
    input jsonb, output jsonb, ms int, created_at timestamptz not null default now());

```

Triggers and functions (0002_logic.sql)

```

create function is_valid_transition(f request_status, t request_status) returns boolean
language sql immutable as $$
    select (f, t) in (
        ('waitlisted','pending'),('waitlisted','approved'),('waitlisted','cancelled'),('waitlisted','expired'),
        ('pending','approved'),('pending','rejected'),('pending','expired'),('pending','cancelled'),
        ('pending','bumped'),
        ('approved','checked_in'),('approved','auto_released'),('approved','cancelled'),('approved','bumped'),
        ('checked_in','completed'),
        ('bumped','pending'),('bumped','cancelled'),('bumped','expired'),('bumped','waitlisted'))
    $$;

create function requests_guard() returns trigger language plpgsql as $$
declare r rooms;
begin
    if new.room_id is not null and new.status in ('pending','approved','checked_in') then
        select * into r from rooms where id = new.room_id;
        if not r.is_active then raise exception 'ROOM_INACTIVE'; end if;
        if new.headcount > r.capacity then raise exception 'OVER_CAPACITY'; end if;
        if exists (select 1 from room_blackouts b where b.room_id = new.room_id and b.during && new.during)
            then raise exception 'ROOM_BLACKOUT'; end if;
    end if;
    if tg_op = 'UPDATE' and new.status <> old.status and not is_valid_transition(old.status, new.status) then
        raise exception 'INVALID_TRANSITION % -> %', old.status, new.status;
    end if;
    new.updated_at := now(); return new;

```

```

end $$;
create trigger requests_guard before insert or update on requests for each row execute function
requests_guard();

create function audit_requests() returns trigger language plpgsql as $$
begin
  if tg_op = 'INSERT' or old.status is distinct from new.status
    or old.room_id is distinct from new.room_id or old.during is distinct from new.during then
    insert into audit_log(at, actor_id, entity, entity_id, action, from_status, to_status, details)
    values (coalesce(new.last_action_at, now()), new.last_actor_id, 'request', new.id,
           coalesce(new.last_action, lower(tg_op)),
           case when tg_op = 'UPDATE' then old.status end, new.status,
           jsonb_build_object('room_from', case when tg_op='UPDATE' then old.room_id end,
                              'room_to', new.room_id, 'during', new.during::text));

  end if;
  return new;
end $$;
create trigger audit_requests after insert or update on requests for each row execute function
audit_requests();
-- plus a generic audit trigger on rooms and room_blackouts (resource registration is audited too)

-- Atomic multi-row plan application (Lab apply, disruption rehome, bumping). One transaction,
-- and the deferred exclusion constraint is checked at COMMIT, so swaps A↔B are legal.
create function apply_plan(p_moves jsonb, p_inserts jsonb, p_actor uuid, p_action text, p_at timestampz)
returns void language plpgsql security definer as $$
begin
  update requests r set room_id = m.room_id, during = tstzrange(m.s, m.e),
    status = coalesce(m.status, r.status), offered_alternatives = coalesce(m.offers,
r.offered_alternatives),
    last_actor_id = p_actor, last_action = p_action, last_action_at = p_at
  from jsonb_to_recordset(p_moves) as m(request_id uuid, room_id uuid, s timestampz, e timestampz,
    status request_status, offers jsonb)

  where r.id = m.request_id;
  insert into requests select * from jsonb_populate_recordset(null::requests, p_inserts); -- app supplies id +
created_at
end $$;

```

RLS (0003_rls.sql). Clients only read; the server writes with the service role.

```

alter table rooms enable row level security; -- ... repeat for every table
create function is_admin() returns boolean language sql security definer stable as
  $$ select exists(select 1 from profiles where id = auth.uid() and role = 'admin') $$;
create policy catalog_read on rooms for select to authenticated using (true); -- same for buildings,
departments, room_blackouts, profiles
create policy req_read on requests for select to authenticated using (
  requester_id = auth.uid() or is_admin()
  or exists (select 1 from rooms where rooms.id = requests.room_id and rooms.approver_id = auth.uid()));
create policy notif_read on notifications for select to authenticated using (user_id = auth.uid());
create policy audit_read on audit_log for select to authenticated using (is_admin());
alter publication supabase_realtime add table requests, notifications, audit_log;

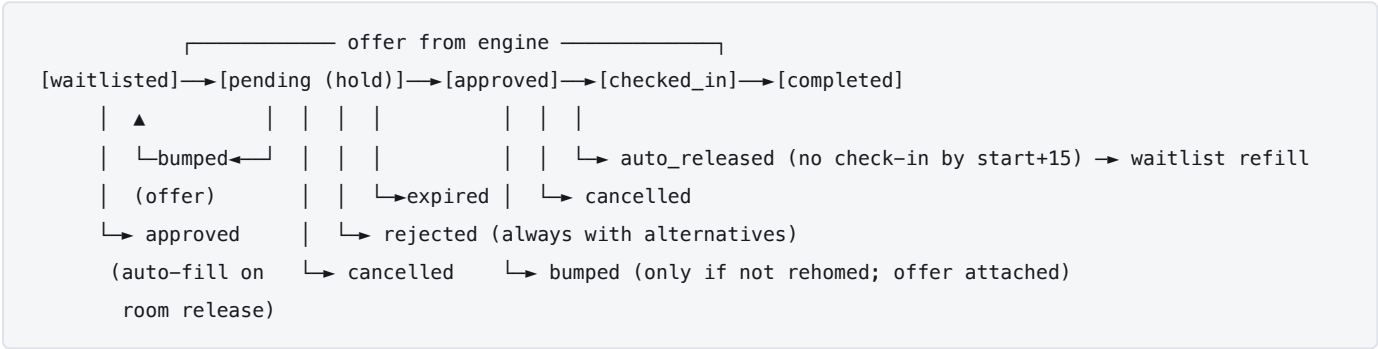
```

Minute-tick job (run once in the SQL editor of each project; **never commit the secret**):

```
select cron.schedule('allotiq-tick', '* * * * *', $$
select net.http_post(url := 'https://<app>.vercel.app/api/jobs/tick',
  headers := jsonb_build_object('x-cron-secret', '<CRON_SECRET>', 'content-type', 'application/json'),
  body := '{} '::jsonb) $$);
```

Priority (server-derived): exam 50 · academic 40 · department_event 30 · meeting 20 · club_event 20 · student_activity 10. "Priority requests" in the metrics means priority ≥ 40.

3. Request state machine



- **Hold:** `pending` claims the slot (the exclusion constraint covers pending). `hold_expires_at = max(now + 10 min, min(now + 120 min, start - 30 min))`.
- **Rehome is not a status change.** When the engine moves a booking to an equivalent room at the same time, the status stays `approved`, the audit row gets `action='rehomed'`, and the user is notified.
- **Bumping policy (decided):** a **pending hold** can be bumped if it gets an alternative offer. An **approved** booking can be moved only if (a) the engine rehomes it to a room that satisfies all its hard constraints at the same time, or (b) the start is more than 24 hours away and it's offered its best alternative. A **checked-in** booking is never bumped.

4. API endpoints (route handlers; bodies validated with Zod schemas from

`src/contracts/api.ts`)

Method + path	Owner	Purpose
POST <code>/api/demo/login {persona}</code>	Aditi	Demo role cards on <code>/login</code> sign in as seeded users
GET <code>/join?t=</code>	Aditi	Judge QR: checks the token, signs in as the demo approver (<code>auth.admin.generateLink({type: 'magiclink'})</code>), redirects to <code>/approvals</code> . Gotcha: add the prod URL to Supabase Auth → Redirect URLs
GET/POST <code>/api/rooms</code> , PATCH <code>/api/rooms/:id</code>	Aditi	Resource registration (admin)
GET <code>/api/rooms/:id/availability?date=</code>	Aditi	Free/busy slots for the day grid
POST <code>/api/rooms/:id/blackouts</code> , DELETE <code>/api/blackouts/:id</code>	Aditi	Maintenance and closure windows
GET <code>/api/rooms/:id/qr</code>	Aditi	QR payload <code>/c/{code}?k={qr_secret}</code>

Method + path	Owner	Purpose
POST /api/ai/transcribe (multipart audio)	Aaditya	Whisper speech-to-text (English)
POST /api/requests/parse {text}	Aaditya	Text → ParsedRequest (Groq strict JSON, or chrono fallback)
POST /api/requests/recommend RequestDraft	Aaditya	Top 3 rooms + breakdowns + "why not" list + alternatives
POST /api/requests {draft, roomId}	Aditi	Creates a pending hold. On 23P01 (overlap) returns 409 + alternatives , never a bare error
GET /api/requests?mine=1, GET /api/requests/:id	Aditi	List, and detail with audit timeline
POST /api/requests/:id/{cancel, accept-offer}	Aditi	Requester actions
GET /api/approvals	Aditi	Approver queue (rooms where approver_id = me)
POST /api/requests/:id/approve	Aditi	pending → approved
POST /api/requests/:id/reject {reason}	Aditi	→ rejected + engine alternatives attached and notified
POST /api/checkin {roomCode, k}	Aditi	QR check-in (window: start-10 min ... start+15 min)
POST /api/admin/requests/:id/simulate-checkin	Aditi	Demo fallback
POST /api/requests/:id/bump-check	Aaditya	Priority request: finds a valid bump/rehome plan
GET /api/lab/scenarios, POST /api/lab/run {scenarioId, solvers[]}	Aaditya	FCFS vs engine (vs ILP) with traces and metrics
POST /api/lab/apply {runId}	Aaditya	Commits the engine plan via apply_plan
POST /api/disruptions/preview {roomId, start, end, reason}	Aaditya	Before/after plan (writes nothing)
POST /api/disruptions/apply {previewId}	Aaditya	Blackout + rehome + offers + notifications, atomically
POST /api/jobs/tick (header x-cron-secret)	Aditi	Expire holds, auto-release no-shows → waitlist refill, complete finished bookings
GET /api/clock, POST /api/admin/clock {advanceMin setTo}	Aditi	Time machine. A jump also runs tick immediately
POST /api/admin/demo/reset	Aditi	Truncate + reseed the demo DB, set the clock to the demo anchor
GET /api/dashboard/{summary, heatmap, ghost, unmet, underused}?from&to&type&building	Nikhil	Utilisation analytics
POST /api/ai/ask {question}	Aaditya	Ask the dashboard (tool calling into analytics functions)

Method + path	Owner	Purpose
GET /api/ai/weekly-insight	Aaditya	Numbers computed in code, phrasing by the LLM
POST /api/ai/explain {requestDraft, recommendation}	Aaditya	Plain-language "why this room"
GET /api/notifications, POST /api/notifications/read	Aditi	In-app bell (Realtime pushes new rows)
GET /api/audit?entityId=	Aditi	Timeline / live feed
GET /api/health	Nikhil	DB ping + Groq model availability + clock

5. Allocation engine (src/engine , pure TypeScript, deterministic)

Contracts (src/contracts/engine.ts , frozen in Phase 0)

```
type ISO = string; type Interval = { start: ISO; end: ISO }; // half-open [start, end)
type Feature = 'projector' | 'mic' | 'computers' | 'smart_board' | 'ac' | 'video_conf' | 'whiteboard' | 'stage' | 'recording';
interface EngineRoom { id; code; buildingId; type: RoomType; capacity; systems; features: Feature[];
  deptId: string|null; access: 'open' | 'dept_only'; hours: { open: 'HH:MM'; close: 'HH:MM'; days: number[] };
  blackouts: Interval[]; booked: { requestId; interval: Interval; priority: number; status; movable: boolean }
[] }
interface EngineRequest { id; requesterId; deptId: string|null; headcount; minSystems; features: Feature[];
  roomType?: RoomType; interval: Interval; priority: number; createdAt: ISO; preferredBuildingId?: string;
  history: Record<string /*roomId*/, number> }
interface EngineContext { rooms: EngineRoom[]; buildings: {id; lat; lng}[]; deptBuilding:
Record<string,string>;
  weights: Weights; now: ISO; tz: 'Asia/Kolkata' }
interface Violation { code: 'CAPACITY' | 'SYSTEMS' | 'FEATURE' | 'TYPE' | 'ACCESS' | 'HOURS' | 'BLACKOUT' | 'OVERLAP';
message: string }
interface ScoreBreakdown { capacityFit; featureMatch; proximity; scarcity; preference; energy; // each 0..1
  weights: Weights; total: number /*0..100*/; wastedSeats: number; notes: string[] }
interface Assignment { requestId; roomId: string|null; score?: ScoreBreakdown; reason?: string }
interface SolveResult { solver: 'fcfs' | 'greedy' | 'bnb' | 'ilp'; assignments: Assignment[];
  metrics: { placed; total; priorityPlaced; priorityTotal; seatsWasted; buildingsActive; objective; ms; nodes
};
  timedOut: boolean; trace: TraceEvent[] }
interface Solver { name: SolveResult['solver'];
  solve(reqs: EngineRequest[], ctx: EngineContext, opts?: { timeoutMs?: number; mustPlace?: string[] }):
SolveResult }
```

Hard constraints (feasibility.ts), which return reasons as well as a yes/no

capacity ≥ headcount · systems ≥ minSystems · requiredFeatures ⊆ room.features · roomType matches if given
· access = dept_only ⇒ req.deptId = room.deptId · the interval falls within operating hours on an open day
(evaluated in IST) · no overlap with blackouts · no overlap with fixed bookings. Overlap is a.start < b.end && b.start
< a.end , so 4–6 and 6–8 **don't** clash.

Soft score (`score.ts`); weights are tunable in `app_settings.engine_weights`

```
capacityFit = 1 - wasted/capacity                                w=0.30
featureMatch = |room.features n niceToHave| / max(1,|niceToHave|)  w=0.10
proximity    = 1 - min(dist(deptBuilding, room.building), 1500m)/1500  w=0.20
scarcity      = 1 - rarity(room) × oversize                        w=0.20  // don't burn big halls on small groups
              rarity = 1 / #rooms with capacity ≥ room.capacity ; oversize = wasted/capacity
preference    = history[room] / max(history)                      w=0.10
energy        = building already active in this slot ? 1 : 0      w=0.10  // packs into fewer buildings → AC-hours saved
total = 100 × ∑ wi · componenti ; notes[] = human reasons ("Same building as CSE dept", "64 systems ≥ 60")
```

Solvers (`solvers/*` , all behind one `Solver` interface, picked with `getSolver(name)`)

FCFS (baseline): for req in sort(reqs, createdAt): pick feasible free room with min wasted seats; else unplaced.

Greedy: sort by (priority desc, #candidates asc, id): take best-scoring free candidate.

Branch & Bound (engine default):

```
cands[r] = feasible rooms for r, sorted (score desc, roomId)      // deterministic
order    = sort reqs by (|cands| asc, priority desc, id)          // most-constrained first
W(p)     = p/10 ; BIG = 1000                                       // lexicographic objective
value(r,c) = W(r.priority)·BIG + c.total/100                       // priority-weighted satisfied first, then fit
best      = greedy(...)                                             // incumbent → always have an answer
suffixUB[i] = ∑j≥i (W(pj)·BIG + max_c value)                     // optimistic bound
dfs(i, used, acc, assign):
  if elapsed > timeoutMs(1500) or nodes > 200k: timedOut = true; return
  if i == n: if acc > best.value: best = snapshot; trace.push(incumbent); return
  if acc + suffixUB[i] ≤ best.value: return                          // prune
  for c in cands[order[i]]: if !overlaps(used[c.room], req.interval):
    used[c.room].push(req.interval); dfs(i+1, ..., acc + value); pop
  dfs(i+1, used, acc, assign u {req: unplaced})                     // leave unplaced
return best (+ metrics, trace)
```

ILP (stretch): $x[r,c] \in \{0,1\}$; $\sum_c x[r,c] \leq 1$; for each room & overlapping pair (r_1,r_2) : $x[r_1,c]+x[r_2,c] \leq 1$; $\max \sum \text{value}(r,c) \cdot x[r,c] \rightarrow \text{HiGHS wasm}$. Same interface; the Lab shows both returning the identical plan.

Required fixture (`__tests__/fixture.test.ts`)

Rooms A (120, projector), B (60, projector), C (80, no projector). Requests, all 4–6 PM, in submission order: Coding Club 50 (no projector), Workshop 55 (projector), AI Club 100 (projector).

- **FCFS (tightest fit):** Coding→B, Workshop→A, AI Club→X, so **2/3**.
- **Engine:** Coding→C, Workshop→B, AI Club→A, so **3/3**.

More tests: `clash8` (FCFS $\leq 5/8$, engine 8/8, fewer seats wasted, fewer buildings); half-open boundaries; determinism (same input 10x gives an identical plan); timeout (1 ms returns the valid greedy plan with `timedOut=true`); access rules; blackouts; bump; rehome.

Conflict alternatives (alternatives.ts), so there's never a bare "rejected"

```
sameRoomOtherSlot(req, room): scan same day ±, then next 3 days, 30-min steps, same duration,
    within hours, free → return 3 nearest to original start
similarRoomSameSlot(req): feasible free rooms at req.interval ranked by score → top 3
```

Priority bumping (bump.ts), following the chosen policy

```
tryBump(R):                                     // R has no free feasible room
  for X in rooms feasible for R if movable bookings are ignored (by score):
    blockers = active bookings in X overlapping R.interval
    if any blocker.priority ≥ R.priority or blocker.status = checked_in: continue
    sub = bnb.solve(blockers, ctx + {R→X}, { mustPlace: blockers })    // same-time rehome
    if sub places all: plans += {R→X, moves: sub, kind:'rehome'}
    else if every unplaced blocker is pending or starts > 24h away:
      plans += {R→X, bumped: blockers with bestAlternative(), kind:'bump_with_offer'}
  return plan preferring 'rehome', then minimal score loss           // applied via apply_plan()
```

Disruption rehoming (rehome.ts)

```
preview(room, window): affected = active bookings in room n window (checked-in → flagged, not moved)
  ctx' = ctx + blackout - affected ; res = bnb.solve(affected, ctx')
  for u in res.unplaced: alts = alternatives(u, ctx'+res) → 'bumped' with offer, or 'waitlisted'
  return { before, after, moves, unplaced: [{req, alts}], metrics }    // UI: "4 affected · 3 rehomed · 1
  offered 6 PM"
apply: insert blackout + apply_plan(moves) + notify each requester (one transaction for the DB part)
```

Waitlist refill (waitlist.ts), called by tick after an auto-release or cancellation

```
onFreed(room, freedInterval = [now, end]):
  pick waitlisted req with interval ≤ freedInterval, feasible(room, req), max (priority, score, -createdAt)
  → transition waitlisted → approved (room policy: waitlist fills are pre-approved), notify "A room opened up"
```

Explanations (explain.ts), all template-based so they work without the LLM

- "Why this room": the top 3 notes from the breakdown, plus a runner-up comparison ("TP 702 was 2nd: 12 seats short of a snug fit").
- "Why not": each excluded room's first violation ("TP 705: only 48 systems").
- **Counterfactual chain** for the Lab. Diff FCFS against the engine and follow who displaced whom: "FCFS gave Coding Club room B first-come. It didn't need a projector, so the engine moved it to C. That freed B for the Workshop and A for AI Club."

6. Groq layer (optional: the app is fully usable with Groq off)

Models (checked against console.groq.com/docs on 2026-09-29; set via env and re-checked at runtime):

Env	Default	Why
GROQ_MODEL_PARSE	openai/gpt-oss-20b	Supports strict json_schema (constrained decoding) and is fast

Env	Default	Why
GRQ_MODEL_REASON	openai/gpt-oss-120b	Tool calling for "Ask the dashboard"; strict schema for insights
GRQ_MODEL_STT	whisper-large-v3-turbo	Fast, low-cost English speech-to-text

- `/api/health` calls `GET https://api.groq.com/openai/v1/models`. If a configured ID is missing, it falls back to the first available ID in a preference list and logs a warning.
- Gotcha: Groq's structured outputs can't be combined with tool use or streaming. So "ask" runs a tool loop first, then a separate strict-format call.

Guardrails (`src/lib/ai/groq.ts`):

- `import 'server-only'`, so the key can never be bundled into the browser.
- Timeouts: parse 6 s, transcribe 15 s, ask 12 s.
- On 429, honour `retry-after` with at most 2 retries plus jitter.
- Circuit breaker: 3 failures in 60 s skips Groq for 2 minutes.
- Every output is checked with Zod.
- `GRQ_DISABLED=true` kill switch, so we can show the fallbacks live.
- The UI shows an "AI assist offline, using basic parser" chip when Groq is off.

6.1 Parse request: system prompt (abridged)

You convert a campus room-booking request into JSON matching the schema. Output JSON only.
Now: {weekday} {YYYY-MM-DD} {HH:MM}, timezone Asia/Kolkata. Next 14 days: {Mon 2026-10-05, Tue ...}.
Rules: never invent values. If something is not stated use null and add the field name to missing_fields.
"60 systems"/"60 computers" → min_systems=60, required_features includes "computers", and headcount=60 if no headcount given.
Bare hours 1–7 without am/pm are PM (campus hours 08:00–20:00). "2–4" → start 14:00, end 16:00.
A weekday name means its next occurrence. Features vocabulary: {FEATURES}. Buildings: {code: name list}.
purpose: exam | academic (class, lab, lecture) | department_event | club_event | meeting | student_activity.

Schema (`strict: true`; every field is required and nullable, `additionalProperties: false`):

```
{ "type":"object","additionalProperties":false,
  "required":["title","purpose","headcount","min_systems","date","start_time","end_time",
    "required_features","room_type","preferred_building_code","notes","missing_fields","confidence"],
  "properties":{"
    "title":{"type":"string"},
    "purpose":{"type":"string","enum":
["exam","academic","department_event","club_event","meeting","student_activity"]},
    "headcount":{"anyOf":[{"type":"integer"}, {"type":"null"}]},
    "min_systems":{"anyOf":[{"type":"integer"}, {"type":"null"}]},
    "date":{"anyOf":[{"type":"string","description":"YYYY-MM-DD"}, {"type":"null"}]},
    "start_time":{"anyOf":[{"type":"string","description":"HH:MM 24h"}, {"type":"null"}]},
    "end_time":{"anyOf":[{"type":"string"}, {"type":"null"}]},
    "required_features":{"type":"array","items":{"type":"string","enum":
["projector","mic","computers","smart_board","ac","video_conf","whiteboard","stage","recording"]}},
    "room_type":{"anyOf":[{"type":"string","enum":
["lab","classroom","seminar_hall","meeting_room","auditorium"]}, {"type":"null"}]},
    "preferred_building_code":{"anyOf":[{"type":"string"}, {"type":"null"}]},
    "notes":{"type":"string"},
```

```
"missing_fields":{"type":"array","items":{"type":"string"}},
"confidence":{"type":"number"} } }
```

- **Post-validation in code:** date is not in the past, end > start, headcount between 1 and 5000, and the building code exists.
- Fields listed in `missing_fields` or with low confidence are highlighted amber on the review form.
- **Fallback:** `fallback-parse.ts` uses `chrono-node` for dates and times, plus regexes for "N systems/students/people" and feature keywords.

The demo sentence "Need a lab with 60 systems Thursday 2–4 for DBMS lab" must parse to: `{title:"DBMS Lab", purpose:"academic", headcount:60, min_systems:60, date:<next Thu>, start_time:"14:00", end_time:"16:00", required_features:["computers"], room_type:"lab"}`. This is a unit test, run against both the Groq path and the fallback.

6.2 Voice (English)

- The browser records with `MediaRecorder` (webm/opus) and sends it to `/api/ai/transcribe`.
- That calls `whisper-large-v3-turbo` with `language=en` and a `prompt` hint of up to 224 tokens (building codes, "DBMS", "lab", "seminar hall", "systems").
- The transcript fills the text box, and the user can edit it before parsing.

6.3 Explain "why this room" (strict schema)

- Input: the draft summary, the chosen room, the breakdown components and notes, and the runner-up.
- Prompt: "Rephrase for a faculty member. Use ONLY the facts given; do not introduce any number not present in the input."
- Output: `{ headline: string (≤90 chars), reasons: string[] (≤3), tradeoff: string|null }`.
- **Numeric grounding check:** every number in the output must appear in the input. If it fails, use the template text.

6.4 Ask the dashboard (tool calling, never SQL)

- Tools. Args are Zod-validated, and each tool maps 1:1 to an `analytics_*` SQL function:
 - `get_utilization({from,to,room_type?,building_code?,weekday?})`
 - `get_heatmap({from,to,room_type?,building_code?})`
 - `get_ghost_rate({from,to,group_by:'room'|'requester_kind'|'weekday'|'building'})`
 - `get_unmet_demand({from,to,group_by:'capacity_band'|'time_band'|'room_type'})`
 - `get_underused_rooms({from,to,threshold_pct,room_type?,weekday?})`
- System prompt: "Answer questions about campus room utilisation using ONLY these tools. Today is ... ; 'last week' = If the tools can't answer, say so. Never guess numbers. ≤3 sentences."
- The tool loop runs at most 3 rounds. A final strict call returns `{ answer, highlights:[{label,value}], chart_tool_call_id: string|null }`.
- **Chart data comes straight from the tool result, never from LLM text.** The grounding check applies to `answer`.
- Canned example chips: "Which labs are underused on Fridays?", "Where do ghost bookings happen most?", "What demand couldn't we meet last month?"

6.5 Weekly insight

- Code computes the facts (occupancy Δ , top ghost rooms, unmet-demand bands, energy idle hours).
- A strict schema turns them into `{ headline, bullets: string[3..5], recommendation }`, with the grounding check.
- Fallback: a template bullet list.

7. Wow layer (all four, sequenced so none of them endangers the core)

1. **Animated engine replay (Lab):** the FCFS chips drop in one by one, and AI Club hits a red "no room" wall. Press "Run engine" and the chips glide (Motion layout animations) into the optimal lanes. KPI counters tick up, and the counterfactual sentence appears. *Must-have*.
2. **Judges join by QR:** a slide/Lab QR code leads to `/join?t=...`, which puts their phone in as the demo approver (in charge of all Tech Park labs, so scene 1 routes to them). They approve live, and Realtime flips the projector. Fallback: Nikhil's phone, already logged in.
3. **Energy-aware allocation:** the `energy` score term, "Buildings in use: 5 → 3" in the Lab, and "idle building-hours (AC could be off)" on the dashboard. Everything is labelled as an estimate.
4. **Live solver swap (ILP):** *stretch, last item of Phase 4, Aaditya*. Only after the Lab replay works. If it isn't done, we answer "how does it scale?" with the solver-interface slide.

Also built in because the demo depends on them:

- **Demo control drawer** (admin, `Ctrl + .`): virtual clock, `+15 min`, `Jump to 14:16`, `Run jobs now`, `Simulate check-in`, `Groq on/off`, `Reset demo`.
- **Live activity ticker** on admin screens, streamed from `audit_log` via Realtime, so judges see the system reacting.

8. UX (Nikhil). The visual design follows Nikhil's references

- **Visual design:** tokens (colours, type, spacing, radius) and component styling are **taken from the UI references Nikhil provides** before each frontend phase. Below are the functional UX requirements, which hold whatever the look.
- **Projector-safe defaults** until the refs arrive: high-contrast theme and consistent status colours.
 - pending = amber, approved = green, checked-in = blue, rejected = red, expired/auto-released = grey, bumped = violet, waitlisted = slate.
 - KPI numbers 48–64 px tabular figures; body text at least 15 px; chart labels at least 13 px.
- **Kit:** `AppShell` (role-based nav + notification bell + ticker), `KpiTile` (animated number), `StatusBadge`, `ScoreBar` (stacked breakdown), `RoomCard`, `Timeline` (from audit), `DayGrid`, `Heatmap` (CSS grid), `EmptyState`, `AiChip` (online/offline).
- **Layouts:** desktop-first; test at **1366×768 and 1920×1080** with no horizontal scroll. The Lab is 2 columns side by side with a KPI row on top.
- **Mobile-first exceptions:** `/approvals`, `/approvals/[id]` (big Approve/Reject buttons plus reason chips) and `/c/[roomCode]` (check-in).
- **Request flow:** a large text box with a mic button → "Understood" form (amber for missing or low-confidence fields) → 3 recommendation cards (score, breakdown bar, 3 "why" bullets, a "why not X" disclosure) → Submit → status page with a live stepper.
- **Day grid:** rooms as rows and 30-min slots from 08:00–20:00 as columns, filtered by building, type and date. Clicking a free cell pre-fills a request.

9. Seed data (drafted by us; the team corrects the names)

- **Anchor:** "Reset demo" sets the virtual clock to **Wednesday 13:50 IST** in the demo week. All data is generated relative to that anchor, so it works whatever date the demo falls on.
- **Buildings (DRAFT, please verify):**
 - TP: Tech Park, where the School of Computing labs are
 - UB: University Building
 - HT: Hi-Tech Block

- BEL: Basic Engineering Lab
- BT: Biotech Block
- TPGA: Dr. T.P. Ganesan Auditorium
- Approximate coordinates are used for the proximity score.
- **Departments (DRAFT):** C.Tech, NWC, DSBS, CINTEL (School of Computing), ECE, EEE, Mech, Biotech.
- **~36 rooms:** 12 labs (with `systems_count`), 14 classrooms, 5 seminar halls, 4 meeting rooms, 1 auditorium. Room codes are plausible drafts such as "TP 401 Lab"; replace them with real ones.
- **~15 users:**
 - 1 admin (Facilities)
 - 3 approvers, one of whom is the "Judge (demo approver)"
 - Requesters: 5 faculty, 4 clubs (Coding Club, AI Club, Robotics Club, Workshop Cell), 1 department office, 1 student
- **4 weeks of history (~900 requests):** peaks at 10–12 and 14–16, a Friday-afternoon dip, and a ghost rate of about 18% skewed to evening club slots. There are 7 unplaced requests for 100+ seats on weekday evenings. The generator is deterministic (seeded PRNG).
- **Scenarios:**
 - `clash-8` , Thursday 16:00–18:00 (Aaditya): designed so FCFS places $\leq 5/8$ with 2 priority requests unplaced, and the engine places 8/8.
 - `disruption` : Thursday, UB Seminar Hall with 4 bookings; 3 are rehomable and 1 is offered 18:00 (Aditi).
 - `no-show` : Wednesday 14:00–15:30 in a TP lab, not checked in, plus a waitlisted Robotics Club request for 14:30–15:30 that fits the freed time (Aditi).
 - `dbms` : Thursday 14:00–16:00 with 3 good labs free and one clear winner. The request is from a CSE faculty member, so proximity decides.

10. Build plan: phases, per member, in build order

- $\leftarrow$ means "depends on". Build against **mocks from** `src/contracts` until the dependency lands.
- Items within a phase run **in parallel across members**, and top to bottom within a member.
- A phase ends when its checkpoint passes on `main` and the deployed URL. Everyone syncs for 10 minutes at each phase boundary.

Phase 0: Setup and contract freeze (all three together)

- **Nikhil:** create the GitHub repo and add collaborators. Link Vercel. Create the 2 Supabase projects and set env vars in Vercel and `.env.example`.
- **Nikhil (prep before the review):**
 - Scaffold the repo: Next.js + TypeScript + Tailwind + shadcn + Vitest + ESLint + CI workflow.
 - Draft `docs/CONTRACTS.md`, `src/contracts/*.ts`, migrations 0001–0003 and the (red) fixture test from this plan.
- **All three:** review and **freeze** the contracts, migrations, state machine and engine I/O, then merge. After this, contracts change only through a PR all three approve.

Phase 1: Foundations → CP1: contracts frozen · engine passes the fixture · schema live on dev · login works · live URL

Aaditya (Intelligence)	Aditi (Platform)	Nikhil (Experience + deploy)
A1 <code>time.ts</code> + <code>feasibility.ts</code> (with violations)	D1 run migrations 0001–0003 on dev; <code>apply_plan</code> ; generate types	N1 CI green on <code>main</code> ; Vercel preview per PR; <code>/api/health</code>

Aaditya (Intelligence)	Aditi (Platform)	Nikhil (Experience + deploy)
A2 <code>score.ts</code> + weights	D2 Supabase SSR auth, <code>requireRole</code> , <code>/login</code> with demo role cards	N2 design system from Nikhil's refs: <code>tokens</code> + <code>AppShell</code> , <code>KpiTile</code> , <code>StatusBadge</code> , <code>ScoreBar</code>
A3 solvers: FCFS, greedy, B&B + registry	D3 <code>clock.ts</code> + <code>/api/clock</code> + <code>app_settings</code>	N3 seed: buildings/depts/rooms/users + 4-week history generator (taking seed per the brief's rule: Aditi's platform list won't be done this early)
A4 fixture + boundary + determinism + timeout tests 	D4 admin resources CRUD + blackouts UI	

Phase 2: Core request flow → CP2: request → recommend → hold → approve on phone → booked, on the deployed URL

Aaditya	Aditi	Nikhil
A5 <code>alternatives.ts</code>	D5 <code>transition.ts</code> + <code>POST /api/requests</code> (409 → alternatives — A5) + cancel + detail	N4 request flow UI: text box + mic → review form → rec cards → submit (mocks first, then real — A7, A8, D5)
A6 <code>engine-adapter.ts</code> (DB → <code>EngineContext</code>) — D1	D6 approver queue + approve/reject (mobile) — N2	N5 My bookings + status tracker with live stepper — D7
A7 <code>/api/requests/recommend</code> — A6	D7 notifications + Realtime hooks <code>useNotifications</code> / <code>useRequestLive</code>	N6 day-grid availability
A8 Groq client + <code>/parse</code> (strict schema) + chrono fallback + demo-sentence test	D8 <code>/api/jobs/tick</code> (expire, auto-release, complete) + <code>pg_cron</code> on dev	
	D9 QR check-in page + printable QR + simulate check-in	

Phase 3: Adaptive engine and insight → the whole 5-scene demo works (plain, without animations)

Aaditya	Aditi	Nikhil
A9 Allocation Lab UI (static side-by-side + KPIs + Apply) — N2 · clash-8 scenario + test	D10 disruption + no-show scenarios	N7a <code>analytics_*</code> SQL functions — N3
A10 <code>bump.ts</code> , <code>rehome.ts</code> , <code>waitlist.ts</code> + tests — D1 <code>apply_plan</code>	D11 <code>demo/reset</code> endpoint + demo control drawer	N7b dashboard UI: KPI row, heatmap, occupancy bars, ghost rate, unmet-demand cards
A11 disruption preview/apply API + UI; waitlist hook in tick — D8	D12 lifecycle edge cases (bumped offers, accept-offer, hold expiry UX)	N8 deploy <code>allotiq-demo</code> : migrations + seed + <code>pg_cron</code> → prod URL

Phase 4: Wow layer and AI polish → CP3: feature freeze at the end of this phase

Aaditya	Aditi	Nikhil
A12 Lab replay animation + counterfactual sentence	D13 audit timeline + live ticker	N9 polish pass on every screen against the refs (loading/empty/error, AI-offline chip, 1366/1920 check)
A13 voice input (English Whisper)	D14 judge QR join	N10 pitch deck v1 + demo script with timings

Aaditya	Aditi	Nikhil
A14 explain phrasing (grounding check)	D15 bug fixes from integration	
A15 Ask the dashboard — N7a · A16 weekly insight		
A17 stretch : ILP solver swap (only if A12 is done)		

Phase 5: Freeze, bug bash, rehearse (all three)

- **Bug bash**: everyone runs the full demo on the deployed URL from phone and laptop. Fix only; no new features.
- **Seed**: the team corrects building, room and department names.
- **Script**: update the demo script with the real numbers the app shows.
- **Rehearse**: 3 timed rehearsals ($\leq 4:00$ each) with **Reset demo** between runs.
- **Backups**: record a backup video of a clean run and export the deck.
- **Before going on stage**: check the prod DB is awake, warm every page, and test the phone hotspot.

Cross-member dependencies (watch these):

- Everything — Phase 0 contracts
- A6 — D1
- D5's 409 path — A5
- N4 wiring — A7, A8, D5
- D6/A9 UI — N2 kit (use raw shadcn if it isn't ready)
- A10/A11 — D1 `apply_plan` + D8 tick
- N7 — N3 seed
- A15 — N7a SQL functions

If **Phase 3 slips** (the emergency valve from the brief; hopefully unused), cut in this order: Ask the dashboard → weekly insight → voice (keep text) → QR auto-release. Never cut the engine, the Lab, or disruption handling.

11. Risks and mitigations

Risk	Mitigation
Short build window for full scope	Contracts drafted before the team meets. Build against mocks. Merge small and often. Phase checkpoints. The cut order is ready.
New to Supabase: RLS silently blocks queries	Writes go through server routes with the service role. RLS only covers reads and Realtime, with the policies above. If Realtime is silent, a 5-second polling fallback kicks in.
Swaps break the overlap constraint mid-update	The constraint is <code>DEFERRABLE INITIALLY DEFERRED</code> , and all multi-row changes go through <code>apply_plan()</code> in one transaction.
Timezone and boundary bugs	Store <code>timestampz</code> , display IST via <code>@date-fns/tz</code> , use half-open ranges; boundary unit tests.
Groq is down, rate-limited, or a model is deprecated	Fallbacks for everything (chrono parser, templates). Timeouts, 429 backoff, circuit breaker, model IDs in env plus a runtime <code>/models</code> check, and the <code>GROQ_DISABLED</code> switch is rehearsed.
Vercel Hobby cron runs only daily	<code>pg_cron</code> + <code>pg_net</code> every minute. A time-machine jump also triggers tick.
The no-show / expiry demo would need a 15-minute wait	Virtual clock and "Jump to 14:16".

Risk	Mitigation
Demo state drifts between rehearsals	One-click Reset demo back to the anchor.
Venue Wi-Fi fails	Phone hotspot, backup video, pre-opened logged-in tabs.
Judge's phone login friction	QR join token; fallback is our own phone.
Serverless cold start or Supabase pause	Open every page 10 minutes before. Free projects pause only after 7 idle days, but check on demo day.
Merge conflicts	Folder ownership; contracts change only via a 3-approval PR; migrations only by Aditi.
Engine blow-up on large inputs	Greedy incumbent + node limit + 1.5 s timeout. The Lab shows <code>nodes</code> and <code>ms</code> .
Seed names are wrong in front of SRM judges	The team verifies building and room names in Phase 5 before the bug bash.
Frontend waits on design refs	The functional UI is built on projector-safe defaults and re-skinned from the refs in N2/N9; tokens live in one file.

12. Demo script (≈3:45, rehearsed with a timer)

Pre-flight:

- Reset demo, which sets the clock to Wed 13:50.
- Tab 1: Dr. Priya (faculty) at `/r/new`. Tab 2: admin at `/admin/lab`. Tab 3: `/admin/dashboard`.
- The judge QR slide is up. Nikhil's phone is logged in as backup approver. Hotspot on.

#	Time	Driver	What happens	Say
0	0:00–0:20	Nikhil	Title slide	"Campus booking today answers <i>what's free?</i> . Allotiq answers <i>what's the best arrangement for everyone?</i> and keeps it that way when plans change."
1	0:20–1:10	Nikhil	Mic: " <i>Need a lab with 60 systems Thursday 2 to 4 for DBMS lab</i> " → the Understood form fills in → top 3 labs with scores and "why" (same building as C.Tech, 64 systems, a snug fit, and "didn't use the auditorium") → Submit → <i>pending, held for you</i> . The judge approves on their phone and the projector flips to Approved live.	"The AI only fills the form. You confirm it, and the engine decides."
2	1:10–2:10	Aaditya	Lab <code>clash-8</code> : Play FCFS , chips drop, AI Club hits the wall, 5/8 placed, 2 priority unplaced, 140 seats wasted, 5 buildings lit. Run engine , chips rearrange, 8/8, 0 priority lost, 38 seats wasted, 3 buildings . The counterfactual sentence appears. (If built: toggle ILP, identical plan.) Apply.	"First-come-first-served isn't fair; it's just first. Global optimisation places everyone."
3	2:10–2:50	Aditi	Disruption: UB Seminar Hall under maintenance Thursday → preview 4 affected · 3 rehomed · 1 offered 6 PM → Apply → the bell pops on Dr. Priya's tab. The live ticker scrolls.	"No one gets a bare 'cancelled'. Everyone gets a plan."
4	2:50–3:20	Aditi	No-show: the 14:00 TP lab booking isn't checked in → Jump to 14:16 → auto-released → the waitlisted Robotics Club gets the room, and a notification fires. Show the room QR; the judge scans it and checks in (fallback: simulate).	"Ghost bookings are reclaimed automatically."

#	Time	Driver	What happens	Say
5	3:20–3:45	Nikhil	Dashboard: heatmap, ghost-booking rate 18% (simulated month), unmet demand "7 requests for 100+ seats on weekday evenings", idle building-hours. Ask: "Which labs are underused on Fridays?" → answer + chart.	"The database makes double-booking impossible, the engine makes the plan, and AI only makes it easier to talk to."

Fallbacks per scene:

- 1. Pasted text if the mic fails; Groq off → basic parser chip.
- 2. Replay works offline, with no Groq needed.
- 3. Preview is read-only, so it's safe to retry.
- 4. Simulate check-in.
- 5. Canned ask chips.

The Lab and dashboard numbers in the table are illustrative targets for designing the seed. On stage, every number shown must come from the live app; update the script in Phase 5. Say "simulated month" for seeded analytics.

13. Pitch deck outline (6 slides + a Q&A appendix)

- 1. **Allotiq**: tagline, team, live URL, and the judge-join QR.
- 2. **The problem**: FCFS plus static room finders cause clashes, big halls wasted on small groups, ghost bookings, and chaos when a room goes down. Visual: the 3-room fixture where FCFS places 2/3.
- 3. **How Allotiq thinks**: Understand (text or voice) → Optimise (hard constraints + a 6-factor explainable score + global optimum) → Adapt (disruption, no-show, priority bumping with rehomings).
- 4. **Live demo** (5 scenes).
- 5. **Under the hood**:
 - Architecture
 - DB exclusion constraint + state machine enforced by triggers + audit log
 - "The AI never decides" guardrails
 - Solver-agnostic interface (B&B today; ILP/CP-SAT for campus scale)
- 6. **Impact + roadmap**:
 - Lab metrics, ghost reclaim, energy estimate
 - Roadmap: equipment/vehicles/staff (the generic resource model already exists), timetable import, campus-wide CP-SAT, SRM SSO, Tamil and other Indian-language voice input
- **Appendix: judge Q&A**
 - *Doesn't this exist?* Finders filter; we optimise globally and self-heal.
 - *Scale?* Swap in ILP/CP-SAT behind the same interface (show the toggle).
 - *AI misparses?* The user confirms the form, and the fallback parser works with AI off.
 - *Double-booking?* A Postgres exclusion constraint; two simultaneous submits give one success and one 409 with alternatives.
 - *Fairness to clubs?* Priority bands, rehome-first bumping, and the unmet-demand panel makes starvation visible.
 - *Why trust the numbers?* LLM text is grounding-checked, and charts come straight from SQL.

14. Verification

- **Engine**: `npm test` (Vitest) must pass fixture (FCFS 2/3, engine 3/3 with Coding→C, Workshop→B, AI→A), clash-8, boundaries, determinism, timeout, bump and rehome. CI blocks merges on failure.
- **DB guarantees**, run in the Supabase SQL editor:

1. Inserting two overlapping approved requests for the same room fails with `23P01` .
 2. 16:00–18:00 and 18:00–20:00 in the same room succeed.
 3. `approved` → `pending` raises `INVALID_TRANSITION` .
 4. Every status change creates an `audit_log` row.
 5. A booking inside a blackout raises `ROOM_BLACKOUT` .
- **Parser:** the demo sentence produces the expected JSON on both the Groq and the fallback path, plus 10 more English phrasings (typed and spoken).
 - **End to end on the deployed URL** (browser at 1366×768 and 1920×1080, plus a real phone): the 5 demo scenes in order, then **Reset demo**, then again. Check: no horizontal scroll, KPIs readable from 5 m away, Realtime flips within 2 s.
 - **Resilience:** `GROQ_DISABLED=true` and the whole demo still completes. Kill Wi-Fi mid-demo and the hotspot recovers.
 - **Jobs:** with `pg_cron` on the demo project, `cron.job_run_details` shows success every minute. A clock jump expires and releases as expected.